Behavioural Fingerprinting of Honeypot and Flow Telemetry for Returning-Attacker Recognition and Adaptive Firewall Rule Generation

Saksham Mishra, Sahil Mohril, Ramyash Gupta
School of Computer Science Engineering

***Abstract*—Honeypots are increasingly used as the sensing layer of adaptive defence, but most systems either profile attackers without acting on the result or act on flow-level signatures without a behavioural model. We present an end-to-end system, deployed as a containerised web application, that closes this loop on recorded telemetry. Four input adapters map CICFlowMeter CSVs (CIC-IDS2017, CIC-DDoS2019), Cowrie and OpenCanary JSON logs and Zeek `conn.log` files onto one event schema. A label-free view of that schema feeds every detection component, and only the evaluator reads dataset labels. A deterministic replay engine streams slices of these sources at 1–100×. It can split one attack class in time and replay its second part from consistently remapped IP addresses to emulate a returning attacker. Sliding-window extraction computes 49 flow-aware features per source IP and per group of sparse sources; features that a source cannot observe are left undefined rather than set to zero. HDBSCAN discovers behaviour clusters without labels and turns them into fingerprints: a centroid, a MinHash signature over categorical traits, per-episode source sets and an explainable rule-based category. Live windows are matched by a weighted sum of anchored cosine similarity and per-family Jaccard containment. A new episode of a fingerprint whose sources are disjoint from all earlier episodes raises a *returning-behaviour* alert. Matches generate nftables, iptables and Suricata rules with a suggested–approved–active–expired lifecycle, an allowlist invariant and an in-app simulated firewall. We evaluate the system on format-faithful sample files (20–50 records per source, looped by the replay engine), not on the full datasets, and report results measured in the running web application as well as in a headless harness. In the application, the scripted demonstration recognised the returning SSH brute-force attacker from a new address within one 5 s hop (similarity 0.938) and the auto-applied rule dropped 58 of its 63 returning events. An eight-source mixed replay under automatic enforcement produced 805 labelled windows with attack-versus-benign precision 1.000, recall 0.796 and F1 0.886, and a per-behaviour macro F1 of 0.458 online and 0.995 after clustering; confusion matrices show that online errors are attack windows flagged as *unknown anomaly* or *unclassified* before their fingerprint exists, never confusions between attack classes. In the headless sweep the returning attacker was recognised in 40 of 42 replays (mean similarity 0.955), with an alert about 6.8 s after its return. The pipeline processed 9,187 events/s and the simulated firewall 108,849 events/s, and no allowlisted event was ever dropped. The rule-based categoriser labelled only 3 of 7 returning classes correctly, which we trace to rate thresholds invalidated by time compression. All numbers are feasibility evidence, and two scripts regenerate them on the real datasets.**

***Index Terms*—Honeypot, behavioural fingerprinting, attacker profiling, HDBSCAN, MinHash, Isolation Forest, returning attacker, firewall rule generation, CIC-IDS2017, Cowrie, Zeek.**

I. Introduction

A honeypot has no legitimate users, so every contact with it is suspect, which sidesteps much of the false-positive problem that dominates in-line anomaly detection [1], [2]. The field has moved from single emulated hosts to hybrid honeynets, cloned decoys, adaptive shells and LLM-generated interaction [3]–[8], and machine learning is now routinely applied to the data they collect [9], [10]. Four findings from recent work frame this paper. Deterministic rule layers carry most of the measured detection accuracy: in one ablation an LLM working alone fell from 94.23% to 55.77% [10]. Behavioural descriptors such as frequency, duration, diversity and timing outlast payload signatures [11]–[13]. Realism matters more than interaction depth [5], [14]. And evaluation is fragmented by small samples, labels derived from the system under test and missing detection rates [4], [10], [15].

Existing systems also show a division of labour. Deception-focused work makes decoys convincing but leaves analysis to people. Profiling work characterises behaviour but rarely acts on it. Response work acts quickly, but on flow- or signature-level inputs instead of behavioural clusters. Only proposals such as AHIH [4] and autonomous deception frameworks [16] describe the full loop from telemetry to enforcement, and the one evaluated end-to-end framework used public datasets [13].

This paper describes and evaluates a working implementation of that loop, delivered as a web application for a security operations centre (SOC). It operates on recorded honeypot logs and labelled flow datasets instead of live traffic, which makes every run reproducible and keeps ground truth independent of the detector. No packet is ever sent: attack traffic exists only as replayed records, and firewall rules are produced as text and enforced only in an in-application simulation. Our contributions are:

1) A **label-isolated ingestion layer**: adapters for four real formats, a single dataset-label-to-behaviour table that marks behaviours with no labelled source, and a typed boundary that keeps labels out of detection (Section IV).
2) A **deterministic replay engine** with a *returning-attacker* mode that splits an attack class in time and replays its second part from consistently remapped addresses with timestamps shifted to the present (Section V).
3) **Flow-aware behavioural fingerprinting**: 49 features with explicit missing-value semantics, HDBSCAN

clustering, MinHash trait signatures, an explainable rule-based categoriser and an anchored cosine-plus-containment similarity (Sections VI–VII).

4) **Episode-based returning-behaviour detection** tied to rule generation in three formats, with a staged lifecycle, an allowlist invariant and a simulated firewall that shows the effect of enforcement (Sections VIII–IX).
5) A **SOC web application** (FastAPI, React, Docker Compose) that exposes replay control, live telemetry, a fingerprint explorer, explained alerts, a rule workbench and an evaluation view with confusion matrices (Section X).
6) A **reproducible evaluation** measured both in the running application and in a headless harness, with confusion matrices, ablations, throughput measurements and an explicit list of what was not measured (Sections XI–XIII).

II. Related Work

Deception platforms and honeypot design. Recent surveys trace honeypots from rudimentary traps to AI-driven deception systems [3], [17] and review how machine learning is applied to IoT honeypots [9]. Aseres et al. review fifteen years of high-interaction honeypots and propose the HONEYLIFECYCLE and Adaptive HIH frameworks [4]. Morić et al. compare seven open-source honeypots on nine criteria in a segmented testbed [1], and Qurbonaliyeva and Abduraxmanova analyse how decoys attract and hold attackers, recommending the DMZ as the default placement [2]. Dynamic honeynets configure themselves to the surrounding network: Zhang and Shi build Honeyd profiles from K-Means centroids of scanned hosts [18], Bartwal et al. launch behavioural honeypots through a SOAR engine [19], and DYNAMIT clones real hosts chosen by K-Medoids over a hybrid Manhattan–Jaccard distance; its decoys returned the same service sets as their templates under an aggressive Nmap scan, while a stock T-Pot exposed more than 100 conflicting services [5]. Anwar et al. allocate honeypots on an attack graph under uncertainty [20], and Silaen et al. capture post-login behaviour in MS SQL honeypots [21]. Generative and adaptive decoys replace scripted responses: shelLM, HoneyGPT, HoneyLLM and the LLM honeypot of Otal and Canbaz produce shell output with language models [6], [7], [22], [23], while HoneyIoT and TangleNet adapt their responses with reinforcement learning [8], [24]. Zhang et al. show the converse risk, identifying 589 Kippo and Cowrie honeypots among 30,000 Shodan SSH hosts by multi-fingerprinting [14]. Our system does not contribute to decoy realism; it consumes what such decoys log.

Behavioural profiling and attribution. Savage et al. ran high-, low- and no-deception SSH hosts side by side and found that a few addresses dominate session volume [11]. Lim and Hobert reduce about 112,000 SSH sessions to 176 patterns and 18 Ward clusters, 7 of which survive IoC validation [12]. Balogh et al. score attack patterns out of 420 and separate human CTF players (122 points on average) from Internet bots (41) [15]. Beltrán-López et al. aggregate CICFlowMeter flows per attacker into 90 features and infer 24 ontology attributes with F1-macro between 0.63 and 0.999 [13]. Lanka et al. apply AI-driven analysis to honeypot data [25], and Shahzad et al. describe an autonomous deception framework with behaviour-based attribution [16]. We adopt the flow-aggregation view of [13] and the log-transform practice of [11], but cluster with HDBSCAN because the densities of the behaviours we observe differ by orders of magnitude, and we make *re-identification after address rotation* an explicit, measured task.

TABLE I
Coverage of five functional axes

System	A1	A2	A3	A4	A5	Validation
DYNAMIT [5]	•	•	○	○	–	Synthetic nets
CNN–LSTM IDS [26]	○	–	–	•	○	CIC-IDS2017
Honeypot–LLM [10]	○	–	•	•	○	52 observations
Real-time profiling [15]	•	–	•	○	–	CTF, Internet
ICS early IDS [27]	•	–	–	•	–	ICS datasets, PLCs
Deception levels [11]	•	○	•	○	–	3 live hosts
Threat-actor ontology [13]	○	○	•	•	○	Public datasets
This work	○	–	•	•	•	Replayed samples, web app

A1 deception platform; A2 attraction/anti-fingerprinting; A3 profiling; A4 learning; A5 response. • substantial, ○ partial, – not addressed.

Learned detection and LLM analysis. Aparna et al. pair an HTTP/SSH honeypot with a CNN–LSTM detector trained on CIC-IDS2017 and report about 95.6% accuracy [26], a benchmark score rather than a measurement on honeypot data. Pashaei et al. train an early IDS for industrial honeypots with adversarial SARSA and report an accuracy of about 0.99 [27]. The grounded honeypot–LLM pipeline of Oliveira et al. reached 94.23% accuracy on 52 observations, but its deterministic rules alone matched that figure [10], and Beekeeper uses LLM feedback to speed up honeypot analysis [28]. Following the "deterministic first" evidence, our decisions are rule- and similarity-based; an Isolation Forest only proposes candidate fingerprints, and HDBSCAN only proposes structure.

Datasets and evaluation. Evaluation remains the weakest link of this literature: the LLM pipeline rests on 52 observations whose labels share the rule vocabulary of the detector [10], the CTF study on 22 attackers [15], and benchmark results such as [13], [26] come from datasets that contain no deception traffic; Aseres et al. also note the absence of performance benchmarks for high-interaction honeypots [4]. These findings motivate two design choices: the evaluator is the only component that reads labels, and every label passes through one auditable mapping table.

Rule generation and enforcement. The Double-DQN IPS of [29] reports 1.8 ms per-flow inference, SAFIRE feeds edge honeypots into an AI-based IDS whose decisions SDN carries out [30]. We generate rules per fingerprint episode, render them for three platforms, stage them through an approval lifecycle and enforce them only in a simulated firewall.

Table I compares this work with representative systems on five functional axes: deception platform, attraction and anti-fingerprinting, profiling, learning, and response.

III. System Overview

Fig. 1 shows the pipeline, organised in six layers from data source to enforcement. Files in a data folder are indexed by a catalog (labels, counts, time ranges). The replay engine streams selected slices through decoy services that fill in response details. A simulated firewall drops events that match active rules, and the capture layer persists events to SQLite and streams batched updates to a dashboard over WebSocket. Accepted events cross into the detection pipeline as label-free `ObservedEvent` records. Feature extraction, matching, clustering, alerting and rule generation run in one synchronous, deterministic orchestrator. A background thread drives it in real time, while tests and the experiment harness call it directly. The evaluator alone receives full events with labels.

IV. Data Model and Input Adapters

A. Event schema and label isolation

An event is $e = (t, src, dst, svc, proto, \mathbf{f}, \mathbf{h}, \mathbf{a}, \mathbf{c}, y, \kappa)$. It holds timestamps and endpoints; flow totals **f** (duration, forward and backward packets, bytes, SYN/ACK/FIN/RST/PSH counts); HTTP fields **h** (method, path, user agent, status); authentication fields **a** (username, password, attempt, outcome); and content **c** (command, payload snippet). The ground-truth label y and campaign identifier κ are kept verbatim from the source. Any field a source does not provide is `None`. The detection pipeline receives a separate type, `ObservedEvent`, which has no y or κ field, so a component cannot read a label even by mistake; a unit test checks this.

B. Adapters

CICFlowMeter CSV (CIC-IDS2017[1], CIC-DDoS2019[2]). Each row, a bidirectional flow, becomes one flow event. Headers are matched after stripping whitespace, against both the 2017 and the newer CICFlowMeter spellings. `Infinity` and `NaN` become missing. CIC-IDS2017 writes afternoon times on a 12-hour clock without AM/PM, so hours 1–7 are shifted by 12 h. Minute-resolution timestamps are spread evenly across the minute in file order. The variant without source IP and timestamp columns is rejected with an explanatory error. *Cowrie*[3]. Connect, login success and failure, command, download and session-close events are mapped, and source ports are carried from each session's connect event to its later events. *OpenCanary*[4]. Port-scan log types (5001–5005) become probe events whose flags are taken from the key set of the kernel log, and SSH, FTP, HTTP, MySQL and other login types become authentication attempts with an unresolved outcome. Boot and system messages are ignored. *Zeek conn.log*[5]. TSV with a `#fields` header or JSON lines are accepted. Flag counts are derived from the connection `history` string. IoT-23 `conn.log.labeled` files[6], whose last columns are joined by spaces, are split correctly, and their `detailed-label` becomes y.

[1]https://www.unb.ca/cic/datasets/ids-2017.html
[2]https://www.unb.ca/cic/datasets/ddos-2019.html
[3]https://docs.cowrie.org/
[4]https://github.com/thinkst/opencanary
[5]https://zeek.org/
[6]https://www.stratosphereips.org/datasets-iot23

TABLE II
Dataset-label to behaviour mapping (excerpt)

Dataset labels	Behaviour
DoS Hulk, DoS GoldenEye, DDoS (2017), WebDDoS	HTTP flood
DoS slowloris, DoS Slowhttptest	Slowloris
PortScan; PartOfAHorizontal-PortScan (IoT-23)	Port scan (fast)
SSH-Patator / FTP-Patator	SSH / FTP brute force
Web Attack – Brute Force	Web login brute force
Web Attack – XSS, Sql Injection	Web vuln probing
Syn (2019)	SYN flood
DrDoS_*, UDP, UDP-lag, TFTP, LDAP, …	UDP flood / amplification
Bot; C&C*, Okiru (IoT-23)	Botnet / C2
Infiltration, Heartbleed	Infiltration

No data source: crawler, scraper, ICMP flood, stealthy scan, credential stuffing, post-login shell.

Algorithm 1 Deterministic replay step

Input: heap Q of (r, k, e); clock r_c; base b; script Σ
$r_1 \leftarrow r_c + \Delta$; $E \leftarrow \emptyset$
while $Q \neq \emptyset$ **and** $\min(Q).r < r_1$ **do**
  $(r, k, e) \leftarrow \textsc{Pop}(Q)$; $e.t \leftarrow b + r$; $E \leftarrow E \cup \{e\}$
fire every unfired step $s \in \Sigma$ with $s.r < r_1$ ▷ narrate, mode, cluster
$r_c \leftarrow r_1$; **return** E

C. Label mapping and coverage

One table maps normalised dataset labels to 19 behaviour keys (Table II). Behaviours that no supported labelled dataset provides are marked *no data source* instead of being simulated. These are search-engine crawler, aggressive scraper, ICMP flood and stealthy port scan, plus credential stuffing and post-login shell activity, which appear in Cowrie and OpenCanary logs but carry no label; a replay can attach a label to such a slice explicitly.

V. Deterministic Replay Engine

A *selection* names a source, optional label and time filters, an offset o on the replay timeline, a compression factor c (dataset seconds per replay second), a row cap, a sampling stride, an optional label override and an optional loop count. A dataset event at time t_d in a slice starting at t_0 is scheduled at replay time $r = o + (t_d - t_0)/c$ and stamped $t = b + r$, where b is the wall-clock time at which the replay starts, so replayed timestamps are shifted to the present. All selections are merged in a heap keyed by r. The engine advances in fixed steps of $\Delta = 0.25$ s regardless of wall-clock speed (Algorithm 1), so a plan always produces the same event sequence and the same results. A unit test checks that step sizes of 0.1, 1 and 5 s yield identical output. When the processing thread falls behind the requested speed, the replay clock slows down instead of skipping events.

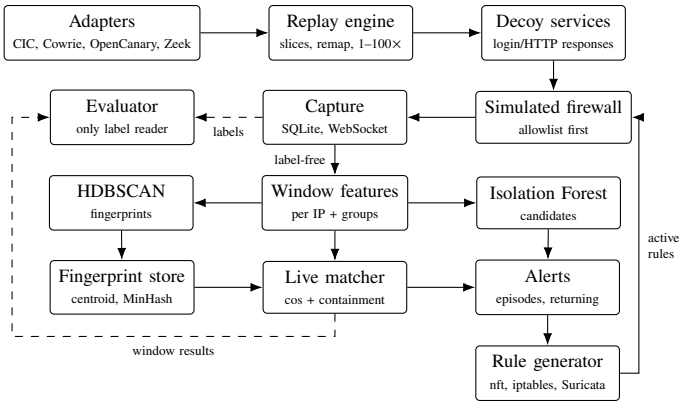


Fig. 1. Architecture. Adapters and replay stand in for the deception and collection layers (L1–L2), feature extraction for L3, HDBSCAN and the fingerprint store for L4, matching and alerts for L5, and rules with the simulated firewall for L6. Dashed edges carry ground truth and window results to the evaluator only.

Algorithm 2 Returning-attacker plan

Input: records R of class ℓ sorted by time; split ϕ; target length T_A; gap g; baseline T_0; seed s

$n_A \leftarrow \lfloor \phi |R| \rfloor$; $A \leftarrow R_{1..n_A}$; $B \leftarrow R_{n_A+1..|R|}$

$m_A \leftarrow \lceil 60/|A| \rceil$, $m_B \leftarrow \lceil 60/|B| \rceil$ ▷ loop counts

$c \leftarrow \text{span}(A, m_A)/T_A$ ▷ shared compression

$o_A \leftarrow T_0$; $o_B \leftarrow o_A + \text{span}(A, m_A)/c + g$

add benign background spanning the plan, looped with fresh IPs if short

add (A, o_A, c, m_A) with campaign κ_A

add $(B, o_B, c, m_B, \text{remap} = \rho_s)$ with campaign κ_B, parent κ_A

script: force clustering at the end of A

Returning-attacker mode (Algorithm 2) takes the records of one attack class in time order and splits them at fraction ϕ. Part A replays with its original addresses after a benign baseline. Part B replays after a gap longer than the episode gap, with every source address a replaced by $\rho_s(a)$: a deterministic, injective-in-practice remapping derived from a keyed BLAKE2b hash with seed s, so all records from one original source keep one new address. Both parts share one compression factor, so their rates stay comparable. Slices too short to fill several windows can be looped back-to-back; benign background loops receive fresh addresses on each copy, so that a handful of hosts does not turn into periodic beacons. Every plan reports when this happened.

VI. Flow-Aware Behavioural Features

Every hop $H = 5$ s, events in the trailing window $W = 10$ s are grouped into *entities*. A source IP becomes an entity when it has at least $n_{\min} = 3$ events in the window or at least $4n_{\min}$ events in a 300 s history. Otherwise its events join a *campaign-cluster group* keyed by traffic shape: protocol, service, well-known port and a request class (SYN-only, RST, established, authentication, HTTP page or sensitive path, command). Groups with at least three events form an entity. This is how spoofed SYN floods and distributed credential attacks, which produce one or two events per source, become visible. For each entity we compute the 49 features of Table III. Count and rate features are transformed by $\tilde{x} = \log(1 + x)$ [11]. Two examples:

$$\text{half_open} = |\mathcal{T}_{\text{ho}}| \,/\, |\mathcal{T}|, \tag{1}$$

$$\text{reply} = \log\big(1 + \text{mean}_{e \in \mathcal{F}}\, b_e / \max(1, f_e)\big), \tag{2}$$

where $\mathcal{T}$ are the TCP events with flag information, $\mathcal{T}_{\text{ho}} \subseteq \mathcal{T}$ those with SYN but no ACK and no reply packets, and f_e, b_e are the forward and backward packets of the flows $\mathcal{F}$ that report both. Inter-arrival statistics use the window, or the long history when the window has fewer than three events. We use $\text{CV} = \sigma/\mu$ and burstiness $B = (\sigma - \mu)/(\sigma + \mu)$.

Missing values. A feature that a source cannot observe is NaN, not zero. Flag ratios are undefined for Cowrie logins, and HTTP ratios are undefined for CIC flows. After scaling, NaN becomes 0, the baseline, so it contributes nothing to similarity, and the categoriser skips it. Treating "not observed" as "zero" would make, for example, every Cowrie window look like a perfect non-SYN-flood.

Categorical traits. Each entity also carries token sets in ten families: service and event type, top ports and a port-spread bucket, flag shapes, $\log_2$ size buckets, paths, user agents, usernames, passwords, command verbs and command bigrams per session, and payload tokens.

VII. Fingerprinting

A. Scaling

A StandardScaler fitted on the recent window buffer supplies the spread, floored at 0.2 so that near-constant features cannot dominate. The centre is the mean of a benign baseline period (the first 30 s of a replay), so normal traffic sits near the origin:

$$z_j = \text{clip}\Big(\frac{x_j - \mu_j^{\text{base}}}{\max(\sigma_j^{\text{buf}}, 0.2)}, -8, 8\Big), \qquad z_j \leftarrow 0 \text{ if } x_j = \text{NaN}. \tag{3}$$

B. Clustering: HDBSCAN rather than DBSCAN

Behaviour densities differ by orders of magnitude: a flood produces hundreds of near-identical windows, a scan a handful.

TABLE III
NUMERIC FEATURE FAMILIES (49 FEATURES)

Family	Features
Volume	events/s, packets/s, bytes/s, connections/s (log)
Protocol	TCP/UDP/ICMP ratio, SYN ratio, SYN:ACK, RST ratio, half-open ratio
Flow shape	duration, packets/flow, bytes/flow, reply ratio
Packets	size mean, size std, size entropy
Timing	IAT mean, IAT std, CV, burstiness
Breadth	unique dst ports, dst IPs, paths, users, passwords; src-port diversity
Auth	auth ratio, fail rate, fail ratio, success-after-fail, user diversity
HTTP	HTTP ratio, error ratio, UA diversity, path entropy, sensitive-path ratio
Content	command count, command ratio
Service	web, SSH/Telnet, FTP, DNS ratios
Sources	long-horizon ports, events, dst IPs; number of sources; rate per source

Algorithm 3 Clustering pass and fingerprint placement

Input: buffer $\mathcal{B}$; store $\mathcal{S}$; threshold θ
re-categorise fingerprints whose centroid drifted
$U \leftarrow \{w \in \mathcal{B} : w.fp = \varnothing\}$; refit spread on $\mathcal{B}$
for $w \in U$ **do** ▷ absorb leftovers
  $(F, s) \leftarrow \arg\max_{F \in \mathcal{S}_{\text{active}}} \text{sim}(w, F)$; **if** $s \geq \theta$ **then** UPDATE$(F, \{w\})$
$L \leftarrow \text{HDBSCAN}(z(U'), m_{\text{cs}}{=}5, m_{\text{s}}{=}3)$ on the remaining U'
for each cluster C with label ≥ 0 **do**
  $(F, s) \leftarrow \arg\max_{F \in \mathcal{S}} \text{sim}(\bar{C}, F)$ ▷ active, candidate, dismissed
  if $s \geq \theta - 0.05$ **then**
    if F dismissed **then** mark C suppressed
    else UPDATE(F, C); promote F if candidate and $|C| \geq m_{\text{cs}}$
  else $F \leftarrow$ CREATE(C): centroid, spread, tokens, MinHash, category, episode

DBSCAN's single ε either merges sparse groups or splits dense ones. HDBSCAN (the scikit-learn implementation) keeps the most stable clusters across densities, needs no k, and labels leftovers as noise, which remain unassigned and may become anomaly candidates. Every 15 s, the clustering pass in Algorithm 3 runs on unassigned windows only. Before clustering, it re-matches leftover windows against existing fingerprints, so that stragglers from an already-known behaviour do not seed a mixed cluster with a newer one. We found this failure mode during development.

C. Fingerprint model

A fingerprint stores a raw centroid and spread, token counts per family (at most 400), a 128-permutation MinHash signature, importance as z-scores against the baseline, source, ASN and country counters, a list of episodes, a version history and a hit timeline. MinHash uses $h_i(x) = \text{SplitMix64}(\text{hash}(x) \oplus s_i)$ with 128 random seeds. An earlier variant of the form $(ax + b) \bmod p$, with 31-bit coefficients on 32-bit hashes, barely wraps modulo p, so it is close to a monotone map. It estimated a true Jaccard of $1/3$ as 0.16, and the SplitMix64 version passes the same test. Live matches nudge the centroid with an exponential moving average ($\alpha = 0.02$) and add tokens, so fingerprints keep learning.

D. Explainable categoriser

Each behaviour has one or more condition lists over centroid features (e.g. SYN flood: half-open ratio ≥ 0.7; packets/flow ≤ 3; $\geq$20 flows/s; random source ports). The first condition, and any marked as required, must be observable and satisfied. Let E be the observable conditions of a list with n conditions and $M \subseteq E$ those satisfied. The confidence is

$$\gamma = \frac{|M|}{|E|}\left(0.85 + 0.15\frac{|E|}{n}\right), \quad |E| \geq n/2, \tag{4}$$

rounded to four decimals. Without rounding, a 3-of-4 score falls just under the 0.75 cut-off in floating point, which we observed. The best list wins if $\gamma \geq 0.75$. Otherwise the fingerprint is *benign* when the mean $|z|$ of its centroid against the baseline is below 1, and *unknown anomaly* when it is not. The matched and unmatched condition texts are kept as the explanation.

E. Similarity

For a live window w and a fingerprint F with scaled vectors $\mathbf{z}_w, \mathbf{z}_F$:

$$\cos^*(w, F) = \max\left(0, \frac{[\mathbf{z}_w; \eta] \cdot [\mathbf{z}_F; \eta]}{\|[\mathbf{z}_w; \eta]\| \, \|[\mathbf{z}_F; \eta]\|}\right), \ \eta = 2, \tag{5}$$

$$J^{\subset}(w, F) = \frac{1}{|\Phi_w|} \sum_{\phi \in \Phi_w} \frac{|T_w^\phi \cap T_F^\phi|}{|T_w^\phi|}, \tag{6}$$

$$S(w, F) = \frac{w_n \cos^* + w_c J^{\subset}}{w_n + w_c}, \quad w_n = 0.6, \ w_c = 0.4. \tag{7}$$

The constant *anchor* dimension η in (5) makes low-energy, baseline-like vectors point along the anchor instead of in arbitrary directions, while strong behaviours are compared by the direction of their deviation. A replay at a different rate along the same direction therefore keeps a high score. A 10 s window holds a small sample of a fingerprint's traits, so plain Jaccard would penalise it for being small. We therefore use containment (6), averaged over the token families Φ_w present in the window, and keep the MinHash Jaccard estimate for fingerprint-to-fingerprint comparison. When a window has no tokens, $S = \cos^*$. Every match records its explanation: per-feature contributions $z_{w,j} z_{F,j} / (\|\cdot\| \|\cdot\|)$, the most divergent features, containment per family, and known versus new tokens.

VIII. DETECTION, EPISODES AND ALERTS

Algorithm 4 processes each hop. A window whose best active fingerprint scores $S \geq \theta = 0.85$ is a match. Otherwise an Isolation Forest (scikit-learn), trained on the baseline windows (150 trees, contamination 0.03), flags anomalies. An anomalous window joins a similar candidate fingerprint ($S \geq \theta - 0.1$) or creates one, at most three per hop; candidates are promoted when a dense cluster confirms them, or by the user.

Algorithm 4 Live matching, episodes and returning detection

Input: windows $\mathcal{W}$ at time t; active fingerprints $\mathcal{S}$; IF model

for $w \in \mathcal{W}$ **do**
  $(F, s) \leftarrow \arg\max_{F \in \mathcal{S}} S(w, F)$
  if $s \geq \theta$ **then**
    $\epsilon \leftarrow$ current episode of F
    if ϵ exists **and** $t - \epsilon.end \leq g_e$ **then** $\epsilon.I \leftarrow \epsilon.I \cup w.I$
    else open ϵ' with $I = w.I$; $\epsilon'.ret \leftarrow$ condition (8)
    raise RETURNING if $\epsilon.ret$ else MATCH; RULEFOR(F, ϵ)
  else if IF$(z_w) < 0$ **then**
    join or create candidate; raise ANOMALY
  append w to the clustering buffer

Episodes and returning behaviour. An episode of F is a maximal run of matches separated by less than a gap $g_e = 25$ s. When a match opens a new episode ϵ_k with source set I_k,

$$\text{returning}(\epsilon_k) \iff k > 1 \ \wedge \ I_k \cap \bigcup_{j<k} I_j = \varnothing. \quad (8)$$

The behaviour is then known but its sources are not, which is the signature of an attacker who rotated infrastructure [11], [12].

Alerts are keyed by (type, fingerprint, episode) and deduplicated within 120 s. The severity rank of an alert hit n times is

$$\text{sev}(n) = \min\big(\text{sev}_0 + [n \geq 10] + [n \geq 50],\ \text{CRITICAL}\big). \quad (9)$$

Returning alerts start at least at HIGH. Categories that resolve to benign never alert.

IX. Rule Generation and Simulated Enforcement

Each (fingerprint, episode) gets at most one rule, whose kind depends on the category (Table IV). The rule records its fingerprint, reason, creation time, expiry and history. It is rendered as nftables (sets with timeouts, meters), iptables (ipset, `hashlimit`, `connlimit`) and Suricata (`threshold`, `http.uri` and `http.user_agent` content). New sources of the same episode extend the rule instead of creating another. The lifecycle is *suggested→approved→active→expired/rejected*. In suggest mode an analyst approves rules; in auto mode they activate immediately, except unknown-anomaly rules, which are never auto-applied.

Allowlist invariant. Allowlisted CIDRs are removed from every rule target, including later user edits. The simulated firewall checks the allowlist before any rule, including rate limits, so an allowlisted source is never dropped. Rate rules use token buckets: with rate λ and burst 2λ, an event at time t passes if $\min(2\lambda, b + \lambda(t - t')) \geq 1$, where b is the bucket level at the previous event t'. Dropped events are counted per rule and kept out of detection, which lets the dashboard show traffic before and after enforcement. No rule is ever executed on a host.

TABLE IV
RULE STRATEGY PER BEHAVIOUR CATEGORY

Category	Rule kind
SYN / UDP / ICMP flood	SYN rate limit / UDP port limit / ICMP limit
HTTP flood, credential stuffing	IP-set block with timeout
Slowloris	connection limit + source block
Scans, SSH/FTP/web brute force, C2	IP block (IP set above 20 sources)
Web probing, post-login shell, malware	Suricata content + IP block
Unknown anomaly	IP block, suggest-only

X. Implementation and Web Application

A. Software stack and deployment

The backend is a Python 3.11 FastAPI service. Events, fingerprints, alerts and rules are persisted in SQLite (write-ahead logging, buffered writes) through SQLModel, and NumPy, pandas and scikit-learn provide the numerical components. The detection pipeline runs in one background thread under a re-entrant lock, and a WebSocket pushes a batched state update to every client four times per second. The frontend is a React 18 single-page application built with Vite and Tailwind CSS, with Recharts for charts and a virtualised event table. One `docker compose up` starts both services: the backend on port 8000 with the data folder mounted read-only, and the frontend served by nginx on port 5173, which proxies `/api` and `/ws` to the backend. A non-Docker path runs the same two processes from a Python virtual environment and the Vite development server. The backend has 61 unit and integration tests (adapters, label table, replay determinism, features, fingerprinting, returning detection, rules and API), and the frontend has unit tests and a browser smoke test.

B. Safety properties

The application never opens a raw socket or sends a request to any host: replayed records are generated in-process from files, and the dashboard states this on every page. Rules are rendered as text for nftables, iptables and Suricata and can be exported, but no command is executed on the host; the *enforcement* switch only changes the simulated firewall that filters the replayed stream. GeoIP and ASN data come from a bundled offline mock.

C. Views

The application has seven views, which also served as our measurement instrument (Section XI).

- *Replay control* (Fig. 2) lists the indexed data sources with their labels and mapped behaviours, and starts the scripted demonstration, a returning-attacker replay (source, class, split, part-A length, gap, speed) or a custom mix of slices. Pause, resume, stop and 1/10/100× speed apply at any time; the status bar shows replay time, effective speed, rule mode and the latest narration step.
- *Live overview* (Fig. 4) shows key indicators, a stacked chart of events per second coloured by the behaviour family that their source was last detected as, rule activations, traffic dropped by the simulated firewall, source countries from the mock GeoIP, and a live event table.
- *Fingerprint explorer* (Fig. 6) projects windows and fingerprint centroids with PCA or t-SNE, lists fingerprints by status and opens a detail panel with a radar profile,

distinguishing features, traits, episodes and history; fingerprints can be renamed, promoted, dismissed, merged or split.
- *Alerts* (Fig. 5) is a filterable feed in which every alert expands into its explanation: the score breakdown of (7), the strongest shared and most divergent features with their z-scores, containment per token family, and known versus new traits.
- *Firewall rules* (Fig. 3) holds the rule lifecycle, the suggest/auto switch, per-format text with an editor, parameters, history, export and the allowlist.
- *Evaluation* (Fig. 9) shows attack-versus-benign scores, a confusion matrix online and after clustering, per-behaviour scores, ground-truth campaigns with times to detect and to rule, and returning-attacker recognition.
- *Settings* exposes every parameter of Sections VI–IX.

XI. Evaluation Methodology

Research questions. RQ1: Does a fingerprint learned from one part of an attack recognise the same behaviour when it returns from new addresses, and how quickly? RQ2: How accurately are windows classified into behaviours, online and after clustering, and where do errors fall? RQ3: What does automatic enforcement change? RQ4: Which design choices matter (ablations)? RQ5: What does the pipeline cost?

Data. The full datasets were not available when we ran these experiments, so all results use the bundled sample files: two CICFlowMeter CSVs with real CIC-IDS2017 and CIC-DDoS2019 headers (50 and 40 rows), a Cowrie log (28 events), an OpenCanary log (27), an IoT-23-style labelled Zeek TSV (30) and a Zeek JSON log (20). They are *synthesised* in each tool's real format, with labels drawn from the datasets' vocabularies, and the replay engine loops them. Such ground truth has known weaknesses [10], [26]: these labels are independent of the detector, but the data is small, synthetic and repetitive. Looping produces artefacts that real data would not have, such as repeated source ports and perfectly periodic timing.

Two measurement paths. (i) Web application. Results W1–W2 were measured on the deployed Docker stack through its own interface. A script (`paper/tools/capture_site.py`) resets the application, starts a scenario through the same REST endpoints that the Replay view calls, waits for the replay to finish, saves the JSON behind the Evaluation, Overview, Alerts and Rules views, and takes the screenshots shown in this paper with a headless browser. W1 is the scripted demonstration at 10×. W2 is an eight-slice mix of all six files at 100× with the rule mode left on *auto-apply*, as the demonstration leaves it. Running the capture twice gave identical numbers. *(ii) Headless harness.* Experiments E1–E6 call the same pipeline directly, without persistence or the network layer, in the default *suggest* mode, which allows sweeps and ablations (`python -m experiments.paper_experiments`). Both paths use the same evaluator.

Label provenance. Window ground truth is the majority mapped label of the events a window contains. Cowrie and OpenCanary windows are unlabelled and excluded unless a replay overrides their label (W2 labels the Cowrie slice as SSH brute force and leaves OpenCanary unlabelled).

Metrics. Let $C \in \mathbb{N}^{K \times K'}$ be the confusion matrix whose entry C_{ij} counts windows with true behaviour i and detected category j; the detected categories add *unknown anomaly* and *unclassified* (no fingerprint matched) to the behaviours. For behaviour i,

$$P_i = \frac{C_{ii}}{\sum_k C_{ki}}, \quad R_i = \frac{C_{ii}}{\sum_j C_{ij}}, \quad F_i = \frac{2P_iR_i}{P_i + R_i}, \tag{10}$$

macro F1 is the mean of F_i over true behaviours, and accuracy is $\sum_i C_{ii} / \sum_{ij} C_{ij}$. We report *C online* (the category of the window's best fingerprint at detection time) and *after clustering*, which credits windows that HDBSCAN later assigned to a fingerprint, using that fingerprint's current category. For attack-versus-benign detection a window is predicted *attack* when it matched a non-benign fingerprint or was flagged anomalous, which gives TP, FP, FN, TN, precision, recall and F1. We also report the time to detect (first event of a campaign to its first flagged window) and the time to rule (to its first rule); returning-attacker recognition, meaning that a part-B window matched a fingerprint that part-A windows built or matched; whether a returning alert was raised; the similarity and the alert delay; and the share of events dropped. Finally we report throughput and per-stage latency.

Setup. An Intel Core i5-1235U laptop with 15.7 GB RAM, Windows 11, Docker Desktop, Python 3.11.14, scikit-learn 1.5.2 and NumPy 2.1.3; screenshots were taken at 1440×900 CSS pixels with device scale factor 2 in the light theme. Default settings throughout (W = 10 s, H = 5 s, θ = 0.85, w_n:w_c = 0.6:0.4, HDBSCAN m_{cs} = 5, m_s = 3, clustering every 15 s). Every run is deterministic, so variation comes only from the split fraction and the remap seed.

XII. Results

A. W1: Scripted demonstration in the web application

The demonstration replays a benign CIC-IDS2017 background from 600 rotating addresses, part A of the SSH-Patator class from its original address, a switch to auto-apply, and part B from a remapped address (Figs. 2–5). Of 2,123 events, 977 labelled windows were evaluated. The application learned 10 fingerprints, nine of them benign plus *SSH brute force #1* (FP-0010). The alert feed contains five alerts: two low-severity unknown-anomaly candidates raised on benign windows during the baseline, the new-fingerprint alert, a match alert (score 0.996) and the returning-behaviour alert (score 0.938, numeric cosine 0.897, containment 1.000). Part B was recognised in its first hop: its time to detect and to rule were both 5.0 s, against 35.0 s and 30.0 s for part A, which had to wait for clustering. The episode rule R-0002 was auto-applied and dropped 58 of the 63 part-B events (92%); R-0001, suggested before the switch, stayed suggested. Table V and Fig. 8 summarise the scores. Precision is 0.636 because the four baseline anomaly windows are false positives; after clustering, 964 of 964 benign and 12 of 13 SSH windows are attributed correctly.

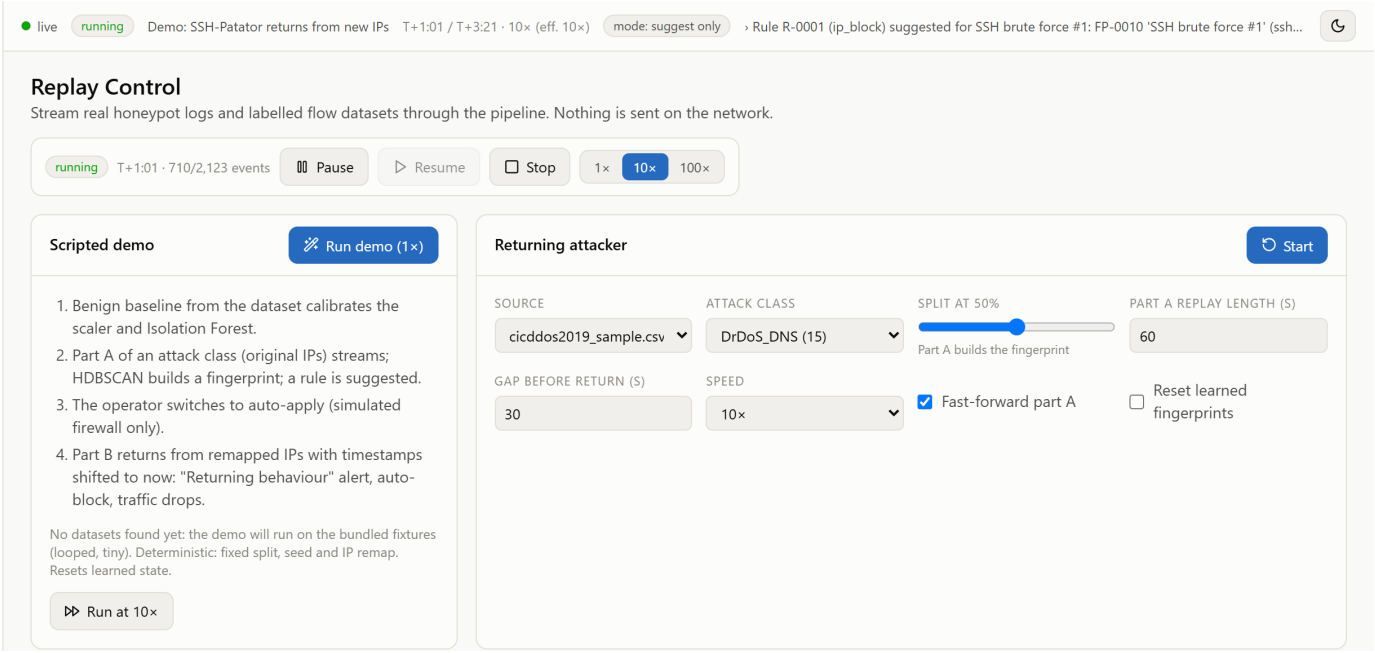


Fig. 2. Replay control during the scripted demonstration at 10× (application screenshot). The status bar narrates the current step; here the rule for the new SSH brute-force fingerprint has just been suggested. The returning-attacker panel configures source, class, split, part-A length, gap and speed.

B. W2: Mixed replay under automatic enforcement

Eight slices from all six files ran overlapping for 236 s of replay time: 7,528 events, of which the simulated firewall dropped 243 (143 SSH brute force including the Cowrie slice, 48 port scan, 34 C2, 18 OpenCanary probes). The application formed 18 fingerprints: nine benign and nine non-benign covering C2, SSH brute force (two, from CIC flows and from Cowrie logins), UDP flood, SYN flood (two), fast port scan, web vulnerability probing (from the unlabelled OpenCanary log) and one unknown anomaly. It raised 17 alerts and generated nine rules, eight active and one unknown-anomaly rule left suggested, as the auto mode requires.

Of 805 labelled windows, attack-versus-benign detection gave TP = 39, FP = 0, FN = 10, TN = 756: precision 1.000, recall 0.796, F1 0.886 (Table V). Fig. 7 shows where the online errors fall. No attack window was ever assigned to a wrong attack class or to benign, and no benign window to an attack. All online errors lie in two columns. *Unknown anomaly* holds 28 attack windows that the Isolation Forest flagged before a fingerprint for their behaviour existed: they count as detected attacks but not as correctly categorised. *Unclassified* holds 147 benign windows and 10 attack windows (9 SSH, 1 C2) seen before the first clustering pass. Online macro F1 is therefore 0.458 at accuracy 0.770. After clustering the matrix is almost diagonal: two windows remain unclassified (one benign, one SSH), giving macro F1 0.995 and accuracy 0.998. Fig. 9 shows the same matrices as the Evaluation view renders them.

Per-campaign timing. Table VI lists the ground-truth campaigns of W2. Every labelled attack campaign was detected within 5–25 s of its first event and received a rule within 20–40 s. Enforcement dropped 27–67% of the events of the brute-force, scan and C2 campaigns. The SYN and UDP flood slices are short (about 28 s and 42 s of replay) and their rate-limit rules became active 35 s and 40 s after their first events, so almost no flood traffic was left to drop. The unlabelled OpenCanary slice received a web-probing rule after 15 s.

TABLE V
W1–W2: Scores measured in the web application

Measure	W1 demo (10×)	W2 mixed (100×, auto)
Events / dropped	2,123 / 58	7,528 / 243
Labelled windows	977	805
Fingerprints (non-benign)	10 (1)	18 (9)
Alerts / active rules	5 / 1	17 / 8
TP / FP / FN / TN	7 / 4 / 6 / 960	39 / 0 / 10 / 756
Attack precision / recall	0.636 / 0.538	1.000 / 0.796
Attack F1	0.583	0.886
Macro F1 online / final	0.785 / 0.980	0.458 / 0.995
Accuracy online / final	0.768 / 0.999	0.770 / 0.998
Mean time to detect / to rule	20.0 / 17.5 s	10.0 / 30.0 s
Returning recognised / alerted	1 of 1 / 1 of 1	–

TABLE VI
W2: Ground-truth campaigns in the web application

Campaign (label)	Behaviour	Events	Dropped	TTD	TTR
CIC-DDoS2019 Syn	SYN flood	3,000	0	5 s	35 s
CIC-DDoS2019 DrDoS_DNS	UDP flood	2,250	0	5 s	40 s
CIC-IDS2017 SSH-Patator	SSH brute force	96	30	10 s	35 s
Cowrie (label override)	SSH brute force	168	113	25 s	20 s
IoT-23 C&C-HeartBeat	Botnet / C2	72	34	10 s	30 s
IoT-23 HorizontalPortScan	Port scan (fast)	180	48	5 s	35 s
OpenCanary (unlabelled)	–	162	18	–	15 s
CIC-IDS2017 BENIGN (480 IPs)	Benign	1,600	0	–	–

TTD/TTR: time from the campaign's first event to its first flagged window / first rule.

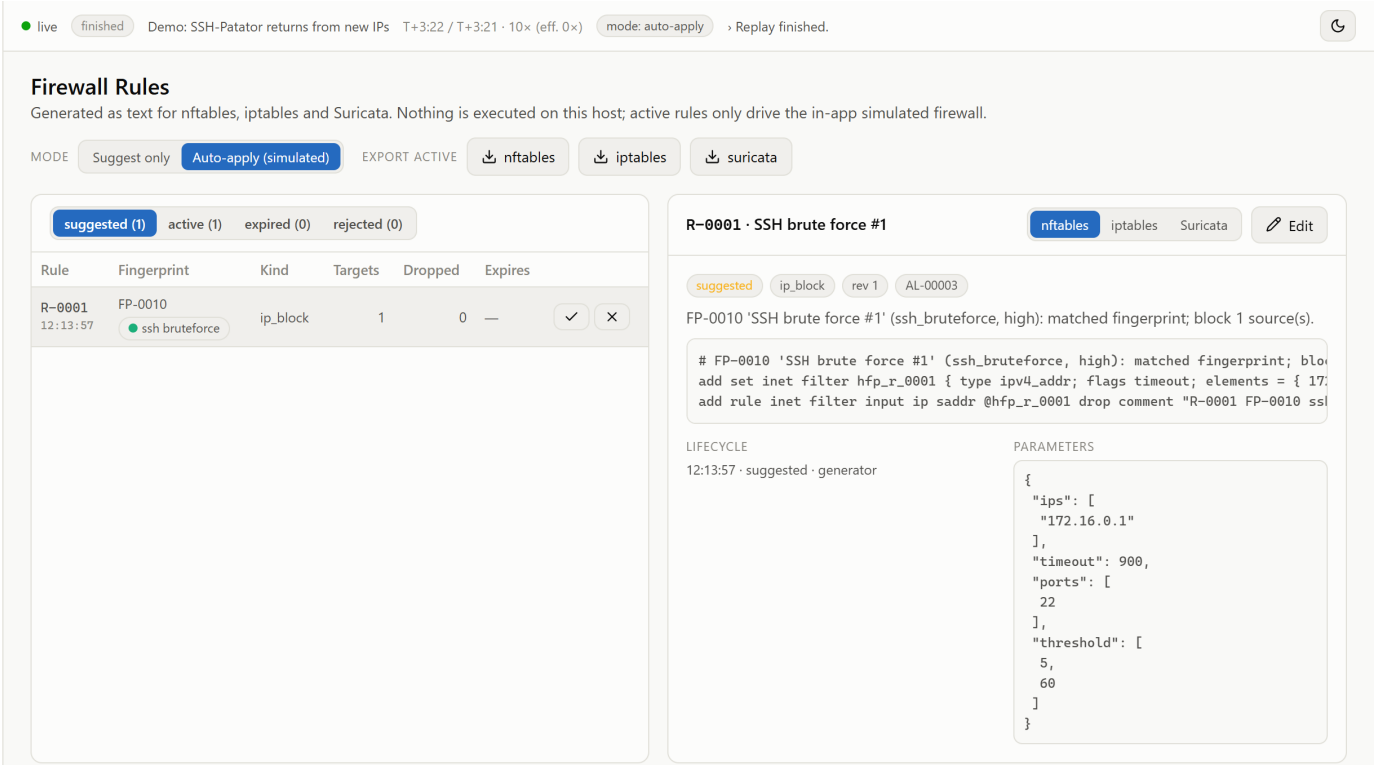


Fig. 3. Firewall rules after the demonstration (application screenshot): R-0001, suggested while the application was in suggest mode, with its nftables rendering and parameters (source, port 22, 900 s timeout, threshold 5 per 60 s); the episode rule R-0002 for the returning source is active. Allowlisted ranges are listed below the rules.

Effect of enforcement (RQ3). The headless run of the same mix in suggest mode (E3 below) evaluated 836 windows with TP = 69 and FN = 11. Under auto-apply, the 243 dropped events removed exactly 31 attack windows (836−805), all of them after a rule had been activated, i.e. windows the system would have matched. The attack windows that remain are disproportionately early ones, observed before any fingerprint existed, so recall falls from 0.863 to 0.796 and online macro F1 from 0.626 to 0.458 while precision stays at 1.000 and the after-clustering macro F1 is unchanged (0.995). Window-level scores of a system that enforces its own decisions therefore understate its effect; the dropped-event counts in Table VI are the complementary measure.

C. E1: Adapter coverage

Table VII shows which fields each format actually provides. Flow sources fill the flow totals almost completely. Honeypot logs fill credentials, commands and HTTP fields but no flow totals, which is why features are defined per field. Parsing costs a fixed 0.1–0.18 s per CSV file, dominated by pandas set-up, and 6–10 ms per JSON or TSV file of this size.

D. E2: Returning-attacker recognition

We ran the returning-attacker plan for every labelled attack class in the samples with split fractions $\phi \in \{0.3, 0.5, 0.7\}$ and two remap seeds, 42 runs in all (Table VIII). Part B was recognised, meaning that it matched the fingerprint built from part A although every source address had changed, in 40 runs, with a returning-behaviour alert in each of them. The mean similarity of the first returning match was 0.955 (range 0.885–0.990), the alert came on average 6.8 s after part B started (one or two hops), and auto-applied rules dropped 87.7% of part-B events on average. Both failures were PortScan with $\phi = 0.3$, where part A contained only 2 of the 8 sample records and never formed a cluster. Part A was first detected 33.8 s and first ruled 30.2 s after it started, on average. Time to rule is shorter than time to detect because a rule is created when clustering forms the fingerprint, while the first *online* detection waits for the next hop.

The categoriser, however, assigned the expected category in only 3 of 7 classes (SSH, FTP and C&C). The returning plan

TABLE VII
E1: FIELD COVERAGE PER ADAPTER (SHARE OF RECORDS WITH THE FIELD)

File	Rec.	Cls.	Pkts	Dur.	Flags	Cred.	Cmd/Path
CIC-IDS2017 CSV	50	6	1.00	0.98	0.80	–	–
CIC-DDoS2019 CSV	40	3	1.00	1.00	0.63	–	–
Cowrie JSON	28	–	–	0.07	0.07	0.54	0.25
OpenCanary JSON	27	–	0.37	–	0.37	0.41	0.22
Zeek TSV (IoT-23)	30	3	1.00	0.40	0.80	–	–
Zeek JSON	20	–	1.00	0.75	1.00	–	–

Cls.: labelled classes. Cred.: username present. Cmd/Path: command (Cowrie) or HTTP path (OpenCanary).

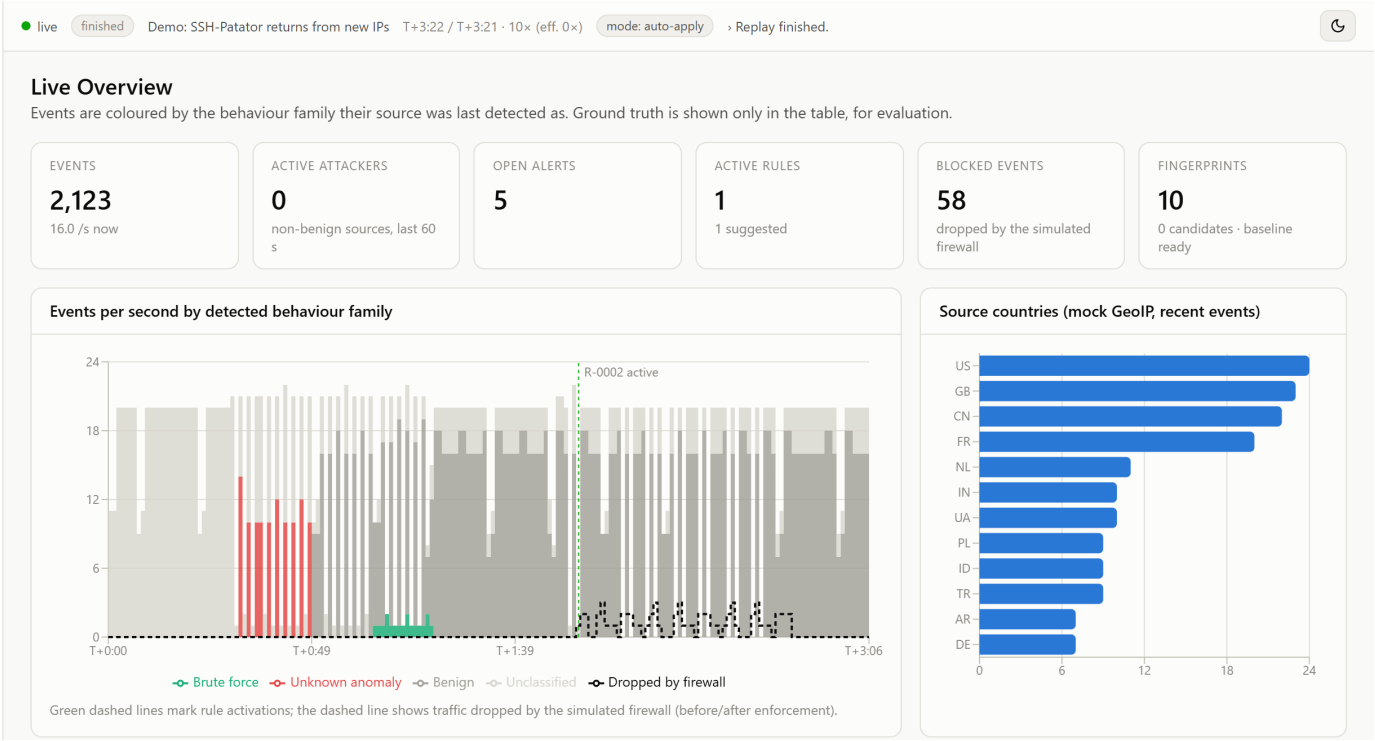


Fig. 4. Live overview at the end of the scripted demonstration (application screenshot). Events per second are stacked by the detected family of their source: grey benign, red unknown-anomaly candidates during the baseline, green SSH brute force once the fingerprint exists. The dashed green line marks activation of rule R-0002 for the returning source, and the black dashed curve the events dropped by the simulated firewall from then on (58 of 2,123 in total).

TABLE VIII
E2: Returning-attacker recognition (6 runs per class)

Class (source)	Rec.	$\bar{S}$ (min)	Delay	Drop	Category
SSH-Patator (2017)	6/6	0.944 (0.885)	5.0 s	0.85	SSH brute force
FTP-Patator (2017)	6/6	0.971 (0.961)	5.0 s	0.92	FTP brute force
PortScan (2017)	4/6	0.974 (0.972)	5.0 s	0.75	C2 (wrong)
Syn (2019)	6/6	0.968 (0.956)	5.0 s	0.91	C2 (wrong)
DrDoS_DNS (2019)	6/6	0.968 (0.957)	16.7 s	0.73	C2 (wrong)
Horiz. scan (IoT-23)	6/6	0.942 (0.928)	5.0 s	0.91	SSH b.f. (wrong)
C&C-HeartBeat (IoT-23)	6/6	0.924 (0.920)	5.0 s	0.92	C2
All	40/42	0.955 (0.885)	6.8 s	0.88	3/7 classes

Rec.: part B matched the part-A fingerprint; every recognised run also raised a returning alert. Delay: part-B start to alert. Drop: share of part-B events dropped.

TABLE IX
E3: Per-behaviour window scores on the mixed replay

		Online		After clustering	
Behaviour	Supp.	Rec.	F1	Rec.	F1
Benign	756	0.806	0.892	0.999	0.999
SSH brute force	39	0.615	0.762	0.949	0.974
Botnet / C2	13	0.615	0.762	1.000	1.000
Port scan (fast)	11	0.455	0.625	1.000	1.000
UDP flood	10	0.300	0.462	1.000	1.000
SYN flood	7	0.143	0.250	1.000	1.000
Macro (P = 1.000)	836	0.489	0.626	0.991	0.995

compresses part A to a fixed 60 s regardless of its original rate, so absolute-rate conditions see rates that the original traffic never had: a looped SYN slice at about one flow per second looks like low-rate periodic beaconing. In the mixed run below, where SYN traffic keeps its sample rate, it was categorised correctly. Recognition does not depend on the category because it uses the similarity (7), but the label shown to an analyst does.

E. E3: Mixed replay, headless in suggest mode

The same eight slices as W2, run headless without enforcement, from all six files produced 7,528 events and 836 labelled windows. They yielded 18 fingerprints, 9 of them non-benign, covering SYN flood, UDP flood, SSH brute force, fast port scan, C2, web probing (from the unlabelled OpenCanary log) and unknown anomaly. Attack-versus-benign detection had TP = 69, FP = 0, FN = 11, TN = 756, giving precision 1.000, recall 0.863 and F1 0.926. Per-behaviour results are in Table IX. Online recall is low because windows observed before the first clustering pass are unmatched by construction. After clustering, the macro F1 was 0.995. The mean time to detect was 10.0 s and to rule 30.0 s. These per-class precisions of 1.0 partly reflect that the sample classes come from different sources and hardly overlap in feature space.

F. E4: Ablations

Table X varies one design choice at a time on the mixed replay and on the seven returning cases ($\phi = 0.5$).

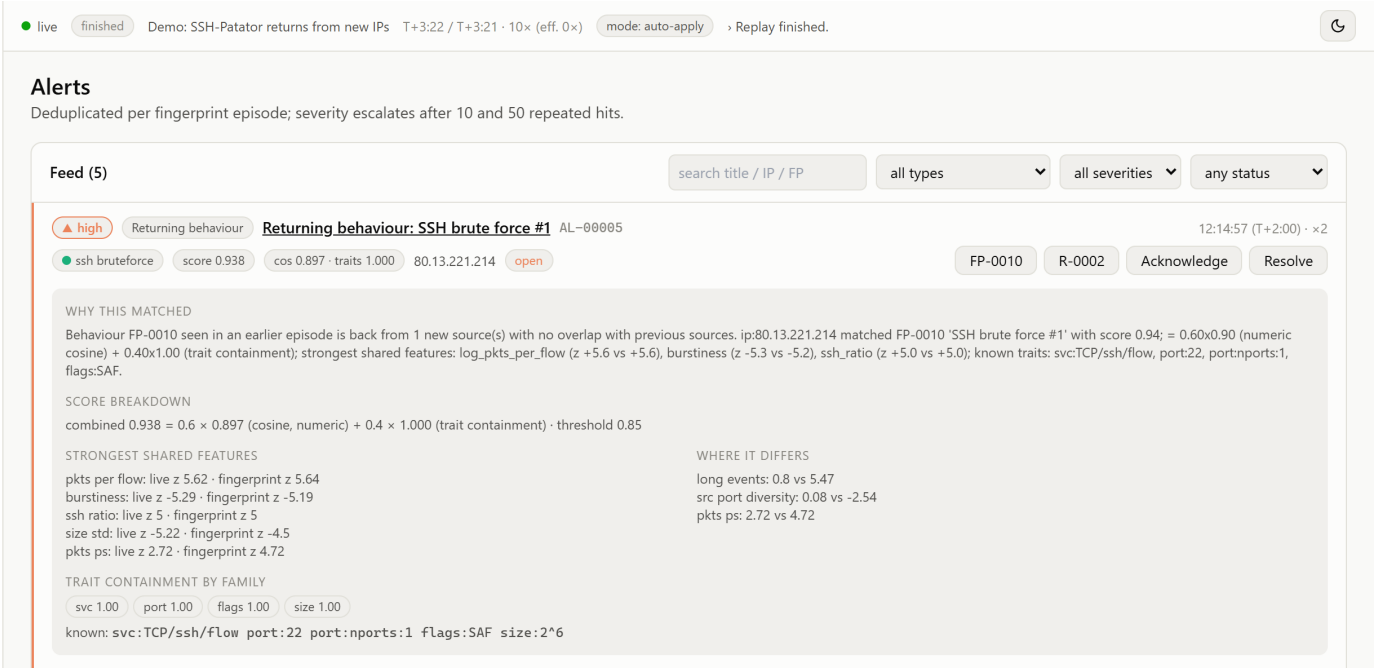


Fig. 5. Alert feed with the returning-behaviour alert expanded (application screenshot). The explanation reports the combined score $0.938 = 0.6 \times 0.897 + 0.4 \times 1.000$, the strongest shared features (packets per flow, burstiness, SSH ratio) with live and fingerprint z-scores, the most divergent features, and trait containment per family. Below it are the earlier match, new-fingerprint and anomaly alerts with their hit counts.



Fig. 6. Fingerprint explorer after the mixed replay (PCA of window features; application screenshot). Diamonds are fingerprint centroids, dots are windows coloured by the family of their fingerprint; benign fingerprints cluster on the left, floods, scans, brute force, C2 and web probing separate to the right.

TABLE X
E4: Ablations (mixed replay; 7 returning cases at $\phi = 0.5$)

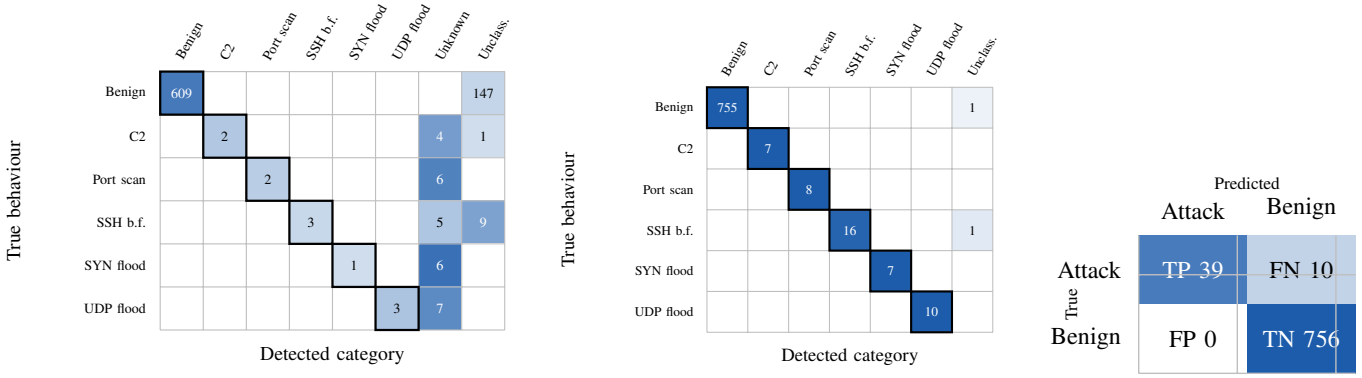
Variant	$F1_{bin}$	$Rec._{bin}$	$mF1_{on}$	$mF1_{fin}$	Ret.
Full (θ=0.85, 0.6/0.4, η=2)	0.926	0.863	0.626	0.995	7/7
Numeric only (w_c=0)	0.912	0.838	0.607	0.991	7/7
Categorical only (w_n=0)	0.940	0.887	0.632	1.000	5/7
No anchor (η=0)	0.926	0.863	0.626	0.995	7/7
$\theta = 0.75$	0.926	0.863	0.626	0.996	7/7
$\theta = 0.90$	0.912	0.838	0.494	0.835	7/7
$\theta = 0.95$	0.873	0.775	0.555	0.980	5/7
DBSCAN $\varepsilon = 0.5$	0.788	0.650	0.438	0.981	7/7
DBSCAN $\varepsilon = 1.0$	0.881	0.787	0.400	0.989	7/7
DBSCAN $\varepsilon = 2.0$	0.881	0.787	0.400	0.989	7/7

Precision was 1.000 in every variant. Ret.: returning attackers recognised.

- *Categorical containment alone* slightly raised mixed-run F1 (0.940) but recognised only 5 of 7 returning attackers. Remapping changes no token, so containment is high for every window of an already-seen service, and the weaker numeric term no longer separated the cases.
- *Numeric cosine alone* recognised all 7 at a slightly lower F1 (0.912). The combination keeps both properties.
- *The anchor dimension* made no measurable difference on this data (mean score 0.961 vs. 0.964); its intended effect concerns low-energy benign windows, which the small benign sample does not exercise.
- *Raising* θ to 0.90 cut the after-clustering macro F1 from 0.995 to 0.835; at 0.95 two returning attackers were missed. Lowering it to 0.75–0.80 changed nothing measurable.
- *DBSCAN* in place of HDBSCAN lowered attack recall from 0.863 to 0.650–0.787 and online macro F1 from 0.626 to 0.400–0.438 for $\varepsilon \in \{0.5, 1, 2\}$, consistent with the density argument of Section VII.

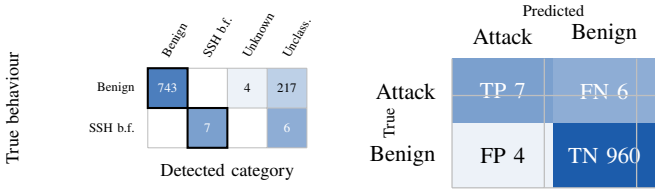
G. E5: Scripted demonstration timeline

Fig. 10 plots the headless run of the W1 scenario at a finer time resolution; its event and drop counts are identical to those of the web application. The deterministic demo replays benign background, SSH-Patator part A, a switch to auto-apply, and part B from new addresses (Fig. 10). The baseline was calibrated on 144 windows at $t = 30$ s. The SSH fingerprint formed at $t = 60$ s, when its rule was *suggested*. Part B started at $t = 110.2$ s, and 4.8 s later the returning alert fired with similarity 0.89, the rule was auto-applied and traffic from the new source began to be dropped (58 of 2,123 events).



(a) Online, at detection time (b) After clustering (c) Attack vs. benign

Fig. 7. W2 confusion matrices measured in the web application (805 labelled windows of the mixed replay under auto-apply). Cells show window counts; shading is row-normalised; outlined cells are correct. Online errors fall only in the *Unknown* and *Unclass.* columns, i.e. before a fingerprint for the behaviour existed; no attack class is confused with another or with benign.



(a) Online (b) Attack vs. benign

Fig. 8. W1 confusion matrices (977 windows of the scripted demonstration). After clustering (not shown) all 964 benign and 12 of 13 SSH windows are correct.

TABLE XI
E6: Measured throughput and latency

Measurement	Result
End-to-end headless pipeline	9,187 events/s (197× real time)
Feature extraction per hop (mean / P95)	6.9 / 18.0 ms
Live matching per hop (mean / P95)	11.6 / 18.0 ms
Clustering pass (mean / P95, 85 runs)	2.7 / 5.0 ms
HDBSCAN, 250 / 500 / 1,000 / 2,000 windows	0.011 / 0.031 / 0.091 / 0.383 s
Simulated firewall, 16 active rules	108,849 events/s
Allowlisted events dropped	0 of 50,000
Rule rendering, three formats	≤0.03 ms per rule
Live API latency under 100× replay	17–72 ms

Four benign windows raised low-severity anomaly candidates, which gives attack-versus-benign precision 0.636 on this small run.

H. E6: Throughput and latency

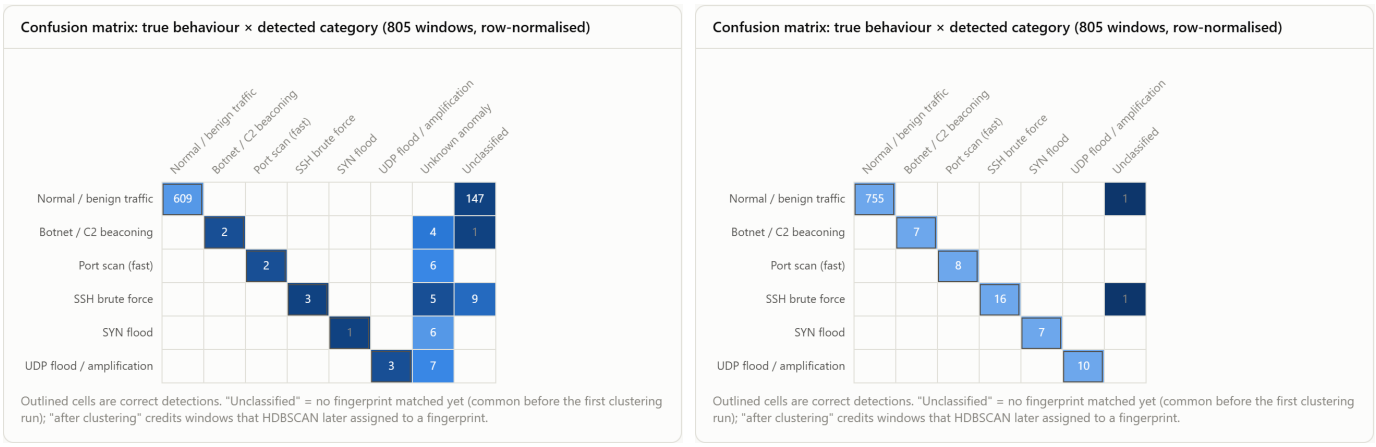
Table XI lists the measured costs. The headless pipeline processed 60,980 events of a 1,306 s replay in 6.64 s, i.e. 9,187 events/s or 197× real time, with feature extraction at 6.9 ms (P95 18.0 ms) and matching at 11.6 ms (P95 18.0 ms) per 5 s hop. HDBSCAN on the window buffer grew from 0.011 s at 250 windows to 0.383 s at 2,000, roughly quadratic in this range, which caps the default buffer at 2,500 windows. With 16 active rules of six kinds, the simulated firewall checked 108,849 events/s; of 200,000 events, 50,000 came from allowlisted sources and none was dropped. Rendering a rule in three formats took at most 0.03 ms. In the live application with SQLite persistence and WebSocket streaming at a requested 100×, the backend processed about 18,600 events in the first second and answered REST requests in 17–72 ms. For orientation only, since hardware and workloads differ: the Double-DQN IPS of [29] reports 1.8 ms per flow, and the grounded LLM pipeline of [10] reports 24.1 s mean inference per assessment.

XIII. Discussion

Behaviour as the unit of response. The returning-attacker result supports the claim that behavioural descriptors survive infrastructure rotation [11], [12]: changing every source address left the fingerprint match intact in 40 of 42 runs, and the episode rule turned that match into an actionable alert within one or two hops. Binding rules to fingerprint episodes gives a unit that is more stable than an address yet still fine-grained enough to choose a rule kind.

Deterministic first. All decisions in the prototype are rule- or similarity-based. The learned components (HDBSCAN and the Isolation Forest) only propose structure, and every decision carries an explanation. This follows the ablation evidence of [10]. Our own ablations point the same way: the categorical term helps separate behaviours but cannot recognise a returning attacker on its own.

Reading the confusion matrices. The matrices of Figs. 7 and 8 separate two questions that a single accuracy figure conflates: *is this window an attack?* and *which behaviour is it?* The first is answered almost immediately (attack precision 1.000 in W2, time to detect 5–25 s), while the second waits for a fingerprint to exist, which takes one clustering pass. The gap between online (0.458) and after-clustering (0.995) macro F1 in W2 is therefore a latency, not a confusion: no attack window was ever assigned to a wrong attack class. The clustering



(a) Online (b) After clustering

Fig. 9. The W2 confusion matrices as rendered by the application's Evaluation view (screenshots).

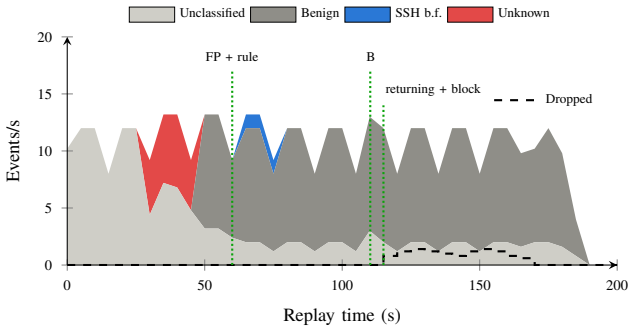


Fig. 10. Demonstration timeline (5 s bins): accepted events per second by the detected category of their source (stacked) and events dropped by the simulated firewall (dashed). Markers: fingerprint and suggested rule (60 s), part B returns from new IPs (110.2 s), returning alert and auto-block (115 s).

interval and the minimum cluster size control this latency; a fingerprint library carried over from earlier runs removes it for known behaviours, which is exactly the returning-attacker case. The diagonal after clustering must also be read against the threats to validity below: the sample classes come from different files and hardly overlap in feature space, and results on public benchmarks do not necessarily transfer to deception data [13].

Evaluating a system that acts. Under auto-apply, blocked traffic no longer produces windows, so recall computed over the windows that remain falls (0.863 to 0.796) although the system did better, not worse. Evaluations of adaptive defences should therefore report enforcement outcomes, such as dropped events per campaign and time to rule, next to detection scores, or score detection in a suggest-mode shadow run, as we do with E3.

Where the categoriser fails. Absolute rate thresholds are fragile under time compression (E2). A deployment on live telemetry does not compress time, but replay-based evaluation does, and so would a deployment whose window size differs from the one the thresholds were written for. Rate conditions should be expressed relative to the baseline, for example as z-scores, or learned from labelled data, with the hand-written rules kept as explanations, as [13] does with SHAP.

Maturity. On a five-level scale from static decoy to closed loop, the prototype implements level-3 analysis (correlated detection, explanations, rule-based response) and parts of level 4: staged rules, an allowlist invariant, fingerprints that update with evidence and drift re-categorisation. It has no adaptive deception, no verification of generated rules against an existing policy and no live traffic.

What was not measured. Unmeasured quantities should be reported next to headline numbers; we did not measure:

- any result on the full CIC-IDS2017, CIC-DDoS2019 or IoT-23 data, or on real Cowrie and OpenCanary logs;
- robustness to an adaptive adversary that shapes its traffic into benign clusters or poisons the buffer;
- end-to-end latency on live traffic, or the effect of enforcement on a real network;
- the consistency of generated rule sets with existing firewall policy;
- the variance of results beyond the split and seed sweep, since every run is deterministic.

Threats to validity. The samples are synthesised, small and looped, so absolute scores, especially the per-class precision of 1.0 and the near-diagonal confusion matrices, overstate what real traffic will yield; the full benchmarks will add label noise and class overlap that the samples do not have [13], [26]. Real CIC data reuses one attacker address across classes, so per-IP windows can mix behaviours. The category ground truth relies on our label table. The thresholds were tuned during development on the same samples. The web-application results are single deterministic runs per scenario.

XIV. Conclusion and Future Work

We presented a reproducible, label-isolated system, deployed as a SOC web application, that turns honeypot logs and labelled flows into behavioural fingerprints, recognises the same behaviour when it returns from new addresses, and

generates staged, allowlist-safe firewall rules in three formats. Measured in the running application on sample data, it recognised the returning attacker of the demonstration within one hop and blocked 92% of its returning events, and on an eight-source mixed replay under automatic enforcement it reached attack precision 1.000 and recall 0.796 with a confusion matrix that is diagonal after clustering. In the headless sweep it recognised returning attackers in 40 of 42 replays, and it ran at about 9,000 events/s. The next steps follow directly from the gaps: running the experiment harness on the full datasets and on live Cowrie, OpenCanary and Zeek feeds; replacing absolute-rate rules with baseline-relative or learned thresholds; adding consistency checks against existing policy and staged rollback before enforcement; carrying fingerprint libraries across runs to close the online categorisation gap; and evaluating against adaptive adversaries.

Ethics and Artifact Availability

No experiment sent traffic to any host: all attack activity consists of replayed records, and generated rules were enforced only in the application's simulated firewall. The source code, the sample files, the experiment harness (`backend/experiments/paper_experiments.py`), the web-capture script (`paper/tools/capture_site.py`), the figure generator (`paper/tools/make_figs.py`) and all raw result files (`paper/results/`) accompany this paper; the datasets themselves must be obtained from their publishers.

References

[1] Z. Morić, V. Dakić, and D. Regvart, "Advancing cybersecurity with honeypots and deception strategies," *Informatics*, vol. 12, no. 1, Art. no. 14, 2025, doi: 10.3390/informatics12010014.

[2] D. Qurbonaliyeva and G. Abduraxmanova, "Analysis of methods of attracting attackers in the honeypot," in *Proc. 8th Int. Conf. Future Netw. Distrib. Syst. (ICFNDS '24)*, 2024, pp. 897–904, doi: 10.1145/3726122.3726253.

[3] S. G. Mahija, "The evolution of honeypots: From rudimentary traps to AI-driven intelligent deception systems," IEEE, 2025.

[4] A. K. Aseres, S. Z. Melese, A. M. Beyene, and L. K. Tegegne, "High-interaction honeypots in cyber deception: Evolution, design, deployment, and future trends," *IEEE Access*, vol. 14, pp. 99861–99896, 2026, doi: 10.1109/ACCESS.2026.3706613.

[5] Y. Maraden, Z. Ananda, I. G. D. Nugraha, and R. F. Sari, "DYNAMIT: K-Medoids-based machine learning for scalable honeynet deception and intelligent threat profiling," *Electronics*, vol. 15, no. 11, Art. no. 2490, 2026, doi: 10.3390/electronics15112490.

[6] M. Sladić, V. Valeros, C. Catania, and S. Garcia, "LLM in the shell: Generative honeypots," in *Proc. IEEE EuroS&PW*, 2024, pp. 430–435.

[7] Z. Wang *et al.*, "HoneyGPT: Breaking the trilemma in terminal honeypots with large language model," arXiv:2406.01882, 2024.

[8] T. Gazdar, L. Murad, S. Al-Jahdali, F. Zwawi, and R. Mandili, "TangleNet: An advanced cyber deception model based on reinforcement learning," *Rev. Rom. Inform. Autom.*, vol. 35, pp. 23–36, 2025.

[9] S. Lanz *et al.*, "Optimizing Internet of Things honeypots with machine learning: A review," *Appl. Sci.*, vol. 15, Art. no. 5251, 2025.

[10] R. Oliveira, T. Gomes, D. Pinho, and F. Almeida, "An integrated honeypot and LLM-based framework for near real-time detection and behavioral analysis of malicious activities," *J. Cybersecur. Priv.*, vol. 6, no. 5, Art. no. 147, 2026, doi: 10.3390/jcp6050147.

[11] E. Savage, N. D. Bastian, and R. Zhou, "Attack behavior observations and profiling through varying cyber deception mechanisms," in *Proc. IEEE Military Commun. Conf. (MILCOM)*, 2025, pp. 758–763, doi: 10.1109/MILCOM64451.2025.11310496.

[12] T. K. Lim and B. Hobert, "Enhancing honeypot-based cyber attribution using hierarchical clustering," *Digital Threats: Research and Practice*, ACM, 2026.

[13] P. Beltrán-López, M. Gil Pérez, and P. Nespoli, "Enhancing strategic decision-making via semantic inference: An adaptive framework for threat actor profiling," *Inf. Fusion*, vol. 134, Art. no. 104359, 2026, doi: 10.1016/j.inffus.2026.104359.

[14] Y.-J. Zhang, W.-J. Liu, K.-N. Guo, and Y.-M. Kang, "Identification of SSH honeypots using machine learning techniques based on multi-fingerprinting," in *Proc. IEEE 6th ITNEC*, 2023, pp. 1376–1381, doi: 10.1109/ITNEC56291.2023.10082467.

[15] Á. Balogh, E. Kail, M. Érsők, and A. Bánáti, "Possibilities for optimization of real time attacker profiling with honeypots," in *Proc. IEEE 19th SACI*, 2025, pp. 111–116, doi: 10.1109/SACI66288.2025.11030113.

[16] F. Shahzad, M. Saleh, and R. N. B. Ujjan, "An autonomous AI-driven cyber deception framework with behaviour-based attribution," *Computers*, 2026.

[17] A. Javadpour, F. Ja'fari, T. Taleb, M. Shojafar, and C. Benzaïd, "A comprehensive survey on cyber deception techniques to improve honeypot performance," *Comput. Secur.*, vol. 140, Art. no. 103792, 2024.

[18] Y. Zhang and Y. Shi, "Constructing dynamic honeypot using machine learning," in *Proc. 8th Int. Conf. Cyber Secur. Inf. Eng.*, 2023, pp. 116–120.

[19] U. Bartwal, S. Mukhopadhyay, R. Negi, and S. Shukla, "Security orchestration, automation, and response engine for deployment of behavioural honeypots," in *Proc. IEEE DSC*, 2022, pp. 1–8.

[20] A. H. Anwar, C. A. Kamhoua, N. O. Leslie, and C. Kiekintveld, "Honeypot allocation for cyber deception under uncertainty," *IEEE Trans. Netw. Service Manage.*, vol. 19, pp. 3438–3452, 2022.

[21] M. Silaen, J. S. Suroso, and C. Lim, "Capturing post-login behaviors in MS SQL honeypots," 2025.

[22] C. Guan, G. Cao, and S. Zhu, "HoneyLLM: Enabling shell honeypots with large language models," in *Proc. IEEE CNS*, 2024, pp. 1–9.

[23] H. T. Otal and M. A. Canbaz, "LLM honeypot: Leveraging large language models as advanced interactive honeypot systems," in *Proc. IEEE CNS*, 2024.

[24] C. Guan, H. Liu, G. Cao, S. Zhu, and T. La Porta, "HoneyIoT: Adaptive high-interaction honeypot for IoT devices through reinforcement learning," in *Proc. 16th ACM WiSec*, 2023, pp. 49–59.

[25] P. Lanka, K. Gupta, and C. Varol, "Intelligent threat detection—AI-driven analysis of honeypot data to counter cyber threats," *Electronics*, vol. 13, no. 13, Art. no. 2465, 2024.

[26] Aparna S, A. Anilkumar, Mohammed Ashmal P K, and D. S. Kumar, "AI-powered honeypot with intrusion detection system," *Int. J. Eng. Develop. Res.*, vol. 14, no. 2, pp. 744–749, May 2026.

[27] A. Pashaei, M. E. Akbari, M. Zolfy Lighvan, and A. Charmin, "Early intrusion detection system using honeypot for industrial control networks," *Results Eng.*, vol. 16, Art. no. 100576, 2022, doi: 10.1016/j.rineng.2022.100576.

[28] N. Ilg, D. Germek, P. Duplys, and M. Menth, "Beekeeper: Accelerating honeypot analysis with LLM-driven feedback," *IEEE Access*, vol. 13, pp. 168034–168054, 2025.

[29] S. V. Mallikarjunaradhya, R. Pavithra, N. Sanjay, J. C. Sriram, and S. V. S. Yadav, "Adaptive real-time intrusion prevention framework using Double DQN and prioritized experience replay," in *Proc. ICAUC*, IEEE, 2026.

[30] S. Slimani *et al.*, "SAFIRE: An intelligent adaptive security framework for IoT environments using edge honeypots, AI-IDS and SDN," *J. Netw. Comput. Appl.*, 2025/2026.